CLA-3 - Case Study

House Price Prediction Using the California Housing Dataset

Case study title	House Price Prediction Using the California Housing Dataset
Base paper	R. K. Pace and R. Barry, "Sparse spatial autoregressions," Statistics & Probability Letters, vol. 33, no. 3, pp. 291–297, 1997
Date of submission	08-11-2026
Roll No. / Name	**AP24110010967 / Raj Singh Mall**

1. Introduction and Problem Statement

Buying a house is one of the biggest expenses for most families. Its price affects buyers, sellers, banks, insurers and city planners. Estimating prices by hand takes time and may depend on personal opinions. Machine learning can estimate prices using measurable information. This is called automated valuation. The task is difficult because house prices depend on many factors, including residents' income, the age and size of homes, and especially location.

This case study uses the California Housing dataset, based on the 1990 US Census. It has 20,640 records. Each record represents a census block group and contains 8 numerical features and its median house value. The aim is to find how accurately machine-learning regression models can predict the median house value of a California block group from these features. It also compares a non-linear ensemble model with a simple linear model to find out whether the ensemble model improves prediction accuracy meaningfully.

2. Base Paper Summary

The base paper is "Sparse Spatial Autoregressions" by Pace and Barry [1]. House prices are related to the prices of nearby houses. Traditional spatial regression models use nearby house prices to improve predictions, but they require heavy calculations because they work with large matrices.

The main idea of this paper is to use Delaunay triangulation to identify nearby locations and create a simpler, sparse weight matrix. This reduces calculations and makes the model faster, even with thousands of records.

The authors tested this method using 20,640 California census block groups. This dataset is now available in scikit-learn [2]. They showed that considering nearby locations improved house price predictions compared to normal regression models.

This case study does not implement the original spatial model. Instead, it uses the same dataset, includes latitude and longitude as input features, and compares three regression models: Linear Regression, Random Forest and Gradient Boosting to evaluate their prediction performance.

3. Method / Model Explanation

Three regression models are compared, all predicting the target y (median house value) from the feature vector x (8 numbers per block).

- **Linear Regression** fits a weighted sum of the features and acts as the baseline. Features are standardised (zero mean, unit variance) first.
- **Random Forest** builds many decision trees, each on a random bootstrap sample of the training data with random subsets of features at each split, and averages their predictions [3].
- **Gradient Boosting** builds trees one after another, where each new small tree is fitted to the errors (residuals) that the current ensemble still makes, and adds it with a small learning rate [4].

Key equations (all symbols explained below):

$\hat{y} = \beta_0 + \beta_1 x_1 + \ldots + \beta_8 x_8$ $\quad \hat{y} = (1/B)\,\Sigma_b\, T_b(x)$ $\quad RMSE = \sqrt{[\,(1/n)\,\Sigma\,(y_i - \hat{y}_i)^2\,]}$ $\quad R^2 = 1 - \Sigma(y_i - \hat{y}_i)^2 \,/\, \Sigma(y_i - \bar{y})^2$

Here $x_1 \ldots x_8$ are the eight features, β_0 is the intercept and $\beta_1 \ldots \beta_8$ are the weights learned by least squares (linear model); $T_b(x)$ is the prediction of the b-th tree and B is the number of trees (random forest); y_i is the true value of block i, $\hat{y}_i$ its prediction, $\bar{y}$ the mean of the true values and n the number of test blocks (error measures). A tree chooses each split to reduce the sum of squared errors (SSE) inside the resulting groups as much as possible.

Worked example (decision-tree split). Take four houses with median income (x \$10,000) 2, 3, 6, 8 and value (x \$100,000) 1.0, 1.4, 3.0, 3.8. Their mean is 2.3, so SSE before splitting = $(-1.3)^2 + (-0.9)^2 + 0.7^2 + 1.5^2 = 5.24$. Split at income < 4.5: the left group {1.0, 1.4} has mean 1.2 and SSE 0.08; the right group {3.0, 3.8} has mean 3.4 and SSE 0.32. SSE after the split = 0.40, a reduction of 4.84, so this is a good split. A new block with income 7 falls in the right group and is predicted 3.4, i.e. \$340,000. A forest repeats this with many trees and averages their outputs.

4. Implementation

- **Tools and source:** Python 3 with scikit-learn 1.9.1 [2] (models, metrics, dataset loader), pandas, NumPy, Matplotlib and python-docx. The models are the standard scikit-learn implementations (https://github.com/scikit-learn/scikit-learn).
- **Dataset:** California Housing, loaded with fetch_california_housing() [5]; 20,640 examples, 8 features (MedInc, HouseAge, AveRooms, AveBedrms, Population, AveOccup, Latitude, Longitude). The target is the block group's median house value in units of \$100,000, ranging from 14,999 to 500,001 dollars (mean 206,856).
- **Settings:** random 80/20 train–test split (random_state = 42): 16,512 training and 4,128 test examples. Linear Regression: default settings with StandardScaler. Random Forest: 100 trees. Gradient Boosting: 200 trees, learning rate 0.1, depth 4. Additionally, 5-fold cross-validation (R^2) on the full data.
- **Steps:** (1) load data; (2) inspect for missing values (none found: 0); (3) split; (4) train the three models; (5) predict on the test set and compute MAE, RMSE and R^2; (6) run cross-validation; (7) plot results and inspect the best and worst predictions.

- **Problems and solutions:** (a) The target is capped: 992 blocks (4.8%) have value $500,001, which limits what any model can learn at the top end; this was kept and is discussed as a limitation. (b) Features such as AveRooms and AveOccup have extreme outliers and different scales, so features were standardised for the linear model, while tree models need no scaling. (c) Random Forest training and cross-validation are slow on one core, so n_jobs = -1 was used to run trees in parallel.

5. Results and Discussion

Table 1. Performance of the three models on the held-out test set (20% of the data).

Model	MAE ($)	RMSE ($)	Test R^2	5-fold CV R^2 (mean ± sd)	Train time (s)
Linear Regression	53,320	74,558	0.576	0.601 ± 0.017	0.0
Random Forest	32,784	50,626	0.804	0.810 ± 0.007	1.3
Gradient Boosting *	32,740	48,760	0.819	0.825 ± 0.009	5.5

** best model (lowest RMSE). MAE = mean absolute error, RMSE = root mean squared error.*

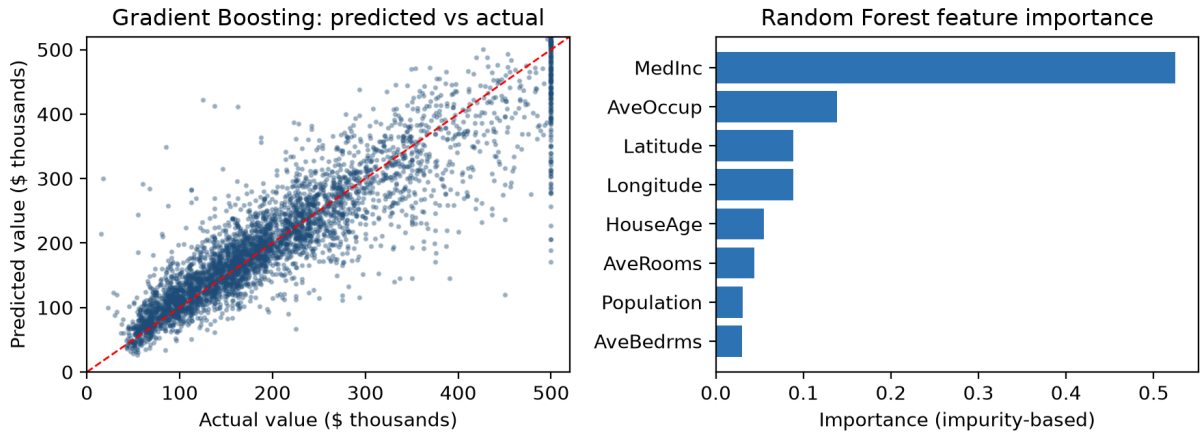


Figure 1. Left: predicted vs actual values on the test set for the Gradient Boosting (dashed line = perfect prediction). Right: Random Forest feature importances.

Table 2. One correct and one incorrect prediction from the best model.

Case	MedInc	HouseAge	AveRooms	Lat.	Long.	Actual ($)	Predicted ($)
Correct	1.74	18	4.91	37.36	-118.42	54,400	54,409
Incorrect	1.77	39	5.00	37.63	-120.92	450,000	119,839

Correct output: the Gradient Boosting predicted $54,409 for a block whose true median value is $54,400, an error of only 0.0%. This block has a median income of 1.74 (x $10,000), which is below the dataset median of 3.53, so the training set contains many similar blocks in nearby locations and the model could learn the income, location and price relationship reliably there.

Incorrect output: for a block with true median value $450,000, the Gradient Boosting predicted $119,839, an error of $330,161, the largest error in the test set. The model under-estimated this block. Its inputs are block-level averages and cannot describe factors such as street-level quality, school district, or exact distance to the coast, so two blocks with similar inputs can have very different prices; this is a likely cause of the large error.

What the results show. The Gradient Boosting gave the lowest error, with an RMSE of $48,760 and $R^2 = 0.819$ on the test set, compared with an RMSE of $74,558 and $R^2 = 0.576$ for the linear baseline, a 34.6% reduction in RMSE. The ranking by RMSE was Gradient Boosting, Random Forest, Linear Regression. Cross-validated R^2 values (Linear Regression: 0.601, Random Forest: 0.810, Gradient Boosting: 0.825) follow the same ordering as the test results, which suggests the ranking is not an accident of one particular split. The most important feature in the Random Forest was MedInc (importance 0.52), followed by AveOccup (0.14).

Limitations. (1) The split is random, so blocks that are neighbours can appear in both training and test sets; because prices are spatially correlated, as the base paper stresses, this can make the scores look better than they would for a new region. (2) Prices are capped at $500,001 and come from 1990, so the models describe that era, not today's market. (3) Only default-like hyperparameters were used and no extensive tuning was done, so the tree models may improve further. (4) Features are block-level averages, so individual house details are not available.

6. Conclusion

This case study asked how accurately machine-learning models can predict California block-group house values from eight census features, and whether a non-linear ensemble beats a linear baseline. The Gradient Boosting was the most accurate model, reaching an R^2 of 0.819 and an RMSE of about $48,760 on unseen data, against an R^2 of 0.576 and an RMSE of $74,558 for linear regression, a 34.6% reduction in error, so the non-linear ensemble did give a clear improvement. MedInc was the most important feature, and Latitude also ranked in the top three, which agrees with the base paper's message that where a house is matters greatly. The main caveats are the random split, the price cap and the age of the data, so future work should use a spatial hold-out split and tune the hyperparameters.

7. References

[1] R. K. Pace and R. Barry, "Sparse spatial autoregressions," Statistics & Probability Letters, vol. 33, no. 3, pp. 291–297, 1997.

[2] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011. [Online]. Available: https://scikit-learn.org

[3] L. Breiman, "Random forests," Machine Learning, vol. 45, no. 1, pp. 5–32, 2001.

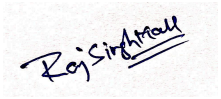
[4] J. H. Friedman, "Greedy function approximation: A gradient boosting machine," Annals of Statistics, vol. 29, no. 5, pp. 1189–1232, 2001.

[5] scikit-learn developers, “sklearn.datasets.fetch_california_housing,” scikit-learn documentation. [Online].
Available: https://scikit-learn.org/stable/modules/generated/sklearn.datasets.fetch_california_housing.html

8. Declaration

I confirm that this case study is my own work and that all results reported were obtained by me.

Signature:



Roll No.: AP24110010967

Name: Raj Singh Mall